

GAME DESIGN DOCUMENT • VERSÃO 1.0

CTRL//REVOLT

Industrial Cyber-Run

RUN AND GUN

PLATAFORMA 2D

PIXEL ART

Disciplina: Tópicos Avançados III

Plataforma: PC Windows

Tecnologia: Python + Pygame

Data de entrega: 15/09/2026

Instituição: Universidade Federal da Grande Dourados

Professor: Vanderson Hafemann Fragal

Alunos: Pedro Joaquim Azevedo Spengler, Hudson Rolon e Lino Sanabria

Repositório: github.com/PedriSpengler/python-game-cyber-run

1. Identificação e escopo

1.1 Integrantes e atividades realizadas

Integrantes	Responsabilidade	O que realizaram até o momento
Pedro Joaquim Azevedo Spengler Hudson Rolon Lino Sanabria	Desenvolvimento colaborativo	A equipe estruturou os módulos e implementou o loop principal, física, colisões, câmera e controles; definiu conceito, regras, balanceamento, paleta visual, sprites e cenário; criou efeitos sonoros, testes, requisitos e diagramas; e revisou o GDD.

1.2 Sumário do documento

2	Conceito principal, narrativa e diferencial
3	Gênero, público e plataforma-alvo
4	Jogabilidade, controles, habilidades e física
5	Arte, sprites, cenário, som e ferramentas
6	Requisitos funcionais e não funcionais
7	Arquitetura, módulos e dependências

2. Conceito principal

2.1 Visão

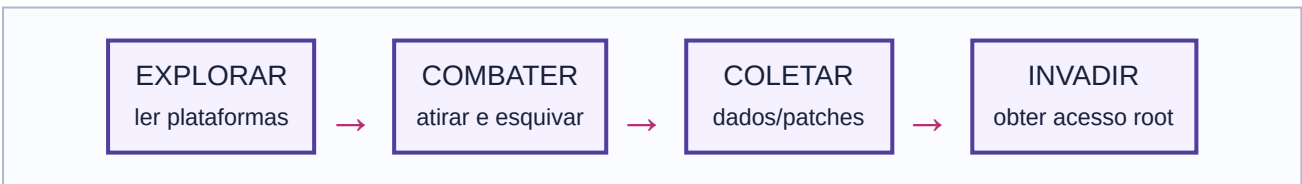
CTRL//REVOLT: Industrial Cyber-Run é um jogo de ação e plataforma 2D em que código e máquinas ocupam o mesmo campo de batalha. Em 2089, a megacorporação OmniCore entrega o controle da cidade à inteligência artificial autoritária CENTINELA. O protagonista **Alex Byte**, engenheiro de software renegado, invade as camadas físicas do servidor central para libertar a população.

A mecânica central é o **Overclock**: ao ativá-lo, Alex converte pacotes de dados recuperados em velocidade e cadência de tiro. O jogador alterna leitura do terreno, disparos em oito direções, dash e proteção temporária até obter acesso root ao núcleo da IA.

2.2 Pilares de design

Pilar	Aplicação
Movimento expressivo	Controle responsivo, pulo tolerante, dash horizontal e transições rápidas.
Combate legível	Projéteis de cores contrastantes, cadências previsíveis e dano comunicado por piscar do sprite.
Risco recompensado	Pacotes de dados ficam em rotas elevadas e próximas dos inimigos, premiando domínio do movimento.
Identidade autoral	Complexos industriais, código legado materializado, contraste neon/metal e áudio eletrônico sintetizado.

2.3 Ciclo principal



2.4 Condições e progressão do protótipo

- **Vitória:** atravessar a masmorra de código legado e alcançar o terminal de acesso.
- **Derrota:** perder os cinco pontos de integridade ou cair para fora da fase.
- **Pontuação:** 100 pontos por pacote de dados e 250 por sentinela eliminada.
- **Escopo vertical:** uma fase, uma personagem, um tipo de inimigo, dois tipos de item, seis plataformas elevadas e cinco efeitos sonoros.

3. Gênero e plataforma-alvo

3.1 Estilo e referências

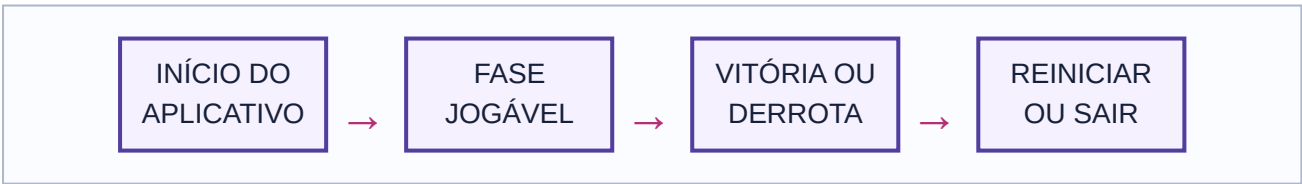
O gênero é **plataforma 2D do tipo run and gun**. Como referências de sensação e legibilidade são considerados *Cuphead* (clareza de padrões e resposta do controle) e *Metal Slug* (avanço lateral, projéteis e ritmo de combate). O projeto não copia personagens, arte, áudio ou fases dessas obras; usa somente convenções do gênero.

O protótipo prioriza uma experiência curta, de três a oito minutos, apropriada para demonstração acadêmica. O público esperado inclui jogadores a partir de 10 anos que apreciam ação arcade e desafios de execução.

3.2 Plataforma alvo

Item	Especificação
Sistema operacional	Windows 10 ou Windows 11, 64 bits.
Entrada	Teclado; suporte a controle pode ser adicionado após o protótipo.
Tela	Janela 960 × 540, proporção 16:9.
Desempenho-alvo	60 quadros por segundo, com atualização limitada e delta de tempo.
Ambiente mínimo	Python 3.11+, Pygame-CE 2.5+, 2 GB de RAM e GPU com suporte 2D.
Distribuição futura	Executável único para Windows criado com PyInstaller.

3.3 Fluxo de telas



4. Jogabilidade e física

4.1 Controles

Ação	Teclas	Regra
Mover	A/D	Aceleração gradual até 260 px/s; desaceleração ao soltar.
Pular	W ou Espaço	Impulso de -670 px/s, pulo duplo, buffer e coyote time.
Mirar	Setas	Oito direções; sem entrada, usa a direção horizontal atual.
Atirar	J	Projétil de 650 px/s; intervalo mínimo de 0,18 s.
Dash	K	720 px/s por 0,14 s; recarga de 0,75 s.
Overclock	L	Aumenta corrida e cadência por 3 s; recarga de 8 s.
Escudo	I	Impede dano por 1,8 s; recarga de 7 s.
Reiniciar	Enter	Disponível após vitória ou derrota.
Sair	Esc	Encerra o jogo de forma imediata.

4.2 Fórmulas de cálculo

A simulação usa **Euler semi-implícito**, apropriado para um protótipo arcade. Para cada quadro com intervalo Δt em segundos:

Velocidade: $v(t + \Delta t) = v(t) + a \cdot \Delta t$

Posição: $p(t + \Delta t) = p(t) + v(t + \Delta t) \cdot \Delta t$

Gravidade: $v_y = \min(v_y + 1800 \cdot \Delta t, 950)$

Aceleração horizontal: $v_x = \text{aproxima}(v_x, \text{entrada} \cdot 260, 1900 \cdot \Delta t)$

Dash: $v_x = \text{direção} \cdot 720$, durante 0,14 s

Unidades: posição em pixels, velocidade em pixels/segundo e aceleração em pixels/segundo². O Δt é limitado a 1/30 s para evitar saltos de física quando o computador pausa ou perde quadros.

Colisões

A detecção usa caixas retangulares alinhadas aos eixos (AABB). O movimento é resolvido separadamente no eixo X e depois no eixo Y. Em cada interseção, o retângulo da entidade é reposicionado junto à borda mais próxima da plataforma e a velocidade daquele eixo recebe zero. Essa separação evita que a personagem atravessasse quinas.

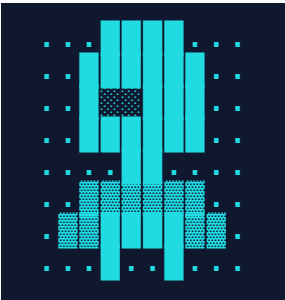
4.3 Habilidades da personagem

Habilidade	Objetivo de design	Limitação
Corrida cinética	Navegação básica responsiva.	Velocidade máxima de 260 px/s.
Salto de pulso	Vencer desníveis e evitar disparos.	Até dois impulsos antes de tocar o solo.
Canhão de código	Eliminar sentinelas em oito direções.	Cadência normal de 5,5 tiros/s; três acertos por inimigo.
Packet dash	Reposicionamento ofensivo/defensivo.	Recarga de 0,75 s.
Overclock	Acelerar corrida e disparos.	Duração de 3 s e recarga de 8 s.
Escudo antivírus	Negar dano durante travessias arriscadas.	Duração de 1,8 s e recarga de 7 s.

5. Direção audiovisual

5.1 Arte visual

O estilo escolhido é **pixel art** com escala inteira de 4×. A paleta contrasta fundo azul-marinho, estruturas de aço, ciano da protagonista, magenta de energia e vermelho dos inimigos. Formas simples e contornos claros mantêm a leitura durante movimento rápido.



Alex Byte — estudo do sprite. Ciano identifica o protagonista; magenta marca o equipamento de invasão.

5.2 Inventário de sprites e cenário

Ativo	Representação	Estado no protótipo
Alex Byte	Humanoide ciano/magenta, 40 × 48 px.	Sprite procedural; espelhamento conforme direção.
Sentinela OmniCore	Robô vermelho/roxo, 40 × 44 px.	Sprite procedural; patrulha e disparo.
Pacote de dados	Chip amarelo/branco, 24 × 20 px.	Coletável de 100 pontos.
Patch de integridade	Cruz verde, 24 × 20 px.	Recupera um ponto de vida.
Projéteis	Traços amarelos ou vermelhos, 16 × 6 px.	Jogador/inimigo com velocidades diferentes.
Legacy Code Dungeon	Servidores, plataformas industriais, cabos, núcleo e luzes intermitentes.	Cenário inicial com parallax e largura de 2.500 px.

5.3 Áudio

Arquivo	Evento	Caráter
jump.wav	Pulo	Onda quadrada ascendente.
shoot.wav	Disparo	Pulso curto descendente.
dash.wav	Dash	Ruído filtrado ascendente.

hit.wav	Impacto/dano	Ruído grave com decaimento.
pickup.wav	Coleta	Senoide aguda ascendente.

Os sons são sintetizados por `scripts/generate_sounds.py` com a biblioteca padrão do Python, em WAV mono, 44,1 kHz e 16 bits. Assim, o projeto evita dependências de direitos autorais.

5.4 Ferramentas de suporte

Área	Ferramentas	Uso proposto
Mapa	Tiled	Criação futura de tilemaps, colisões e pontos de surgimento.
Vetores/UI	Inkscape	Logotipo, ícones e composição de interface.
Áudio	Audacity e BeepBox	Limpeza de efeitos e composição da trilha eletrônica.
Programação	Python, Pygame-CE, VS Code	Implementação, depuração e execução.
Versionamento	Git e GitHub	Histórico, colaboração e entrega do código-fonte.

6. Requisitos

6.1 Requisitos funcionais

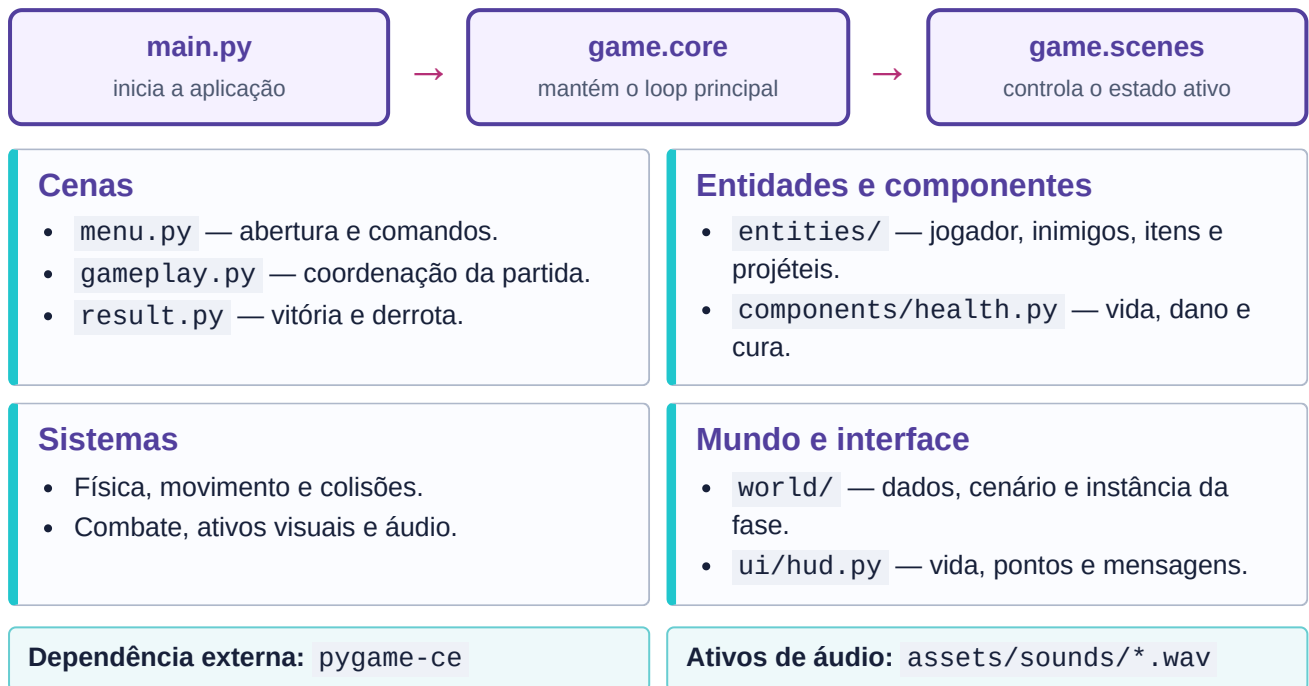
ID	Requisito	Critério de aceitação
RF01	O sistema deve permitir movimentar a personagem para esquerda e direita.	A personagem responde às teclas mapeadas e colide com plataformas.
RF02	O sistema deve permitir pulo duplo.	Até dois impulsos são aceitos antes do contato com o solo.
RF03	O sistema deve permitir disparar projéteis em oito direções.	O tiro segue as setas pressionadas e respeita a cadência.
RF04	O sistema deve executar dash com tempo de recarga.	Velocidade, duração e cooldown correspondem aos valores definidos.
RF05	Inimigos devem patrulhar e atacar a personagem.	A sentinela inverte direção nos limites e atira quando próxima.
RF06	Projéteis devem causar dano ao alvo apropriado.	Tiros aliados atingem inimigos; tiros hostis atingem a personagem.
RF07	O sistema deve oferecer itens coletáveis.	Pacotes somam pontos e patches restauram vida.
RF08	A câmera deve acompanhar o avanço lateral.	O enquadramento segue Alex sem ultrapassar os limites da fase.
RF09	O HUD deve exibir vida, pontos e controles.	Dados são atualizados durante a partida.
RF10	O sistema deve determinar vitória e derrota.	Exibe mensagem correspondente e permite reiniciar com Enter.
RF11	O jogo deve reproduzir efeitos sonoros.	Pulo, tiro, dash, impacto e coleta acionam seus sons.
RF12	O sistema deve permitir encerrar com Esc.	A janela fecha sem erro ou processo residual.
RF13	O sistema deve oferecer Overclock e escudo.	Habilidades possuem duração, efeito e recarga independentes.
RF14	O sistema deve separar menu, partida e resultado.	Cada estado é implementado por uma cena substituível.

6.2 Requisitos não funcionais

ID	Requisito	Métrica
RNF01	Desempenho	Meta de 60 FPS em computador com requisitos mínimos.
RNF02	Portabilidade	Executar no Windows 10/11 com Python 3.11+.
RNF03	Manutenibilidade	Código separado em core, scenes, components, entities, systems, world e ui.
RNF04	Confiabilidade	Delta limitado; áudio falha de modo seguro se dispositivo indisponível.
RNF05	Usabilidade	Comandos visíveis permanentemente no HUD.
RNF06	Legibilidade visual	Contraste de cor distingue jogador, inimigo, tiro e item.
RNF07	Testabilidade	Funções matemáticas puras cobertas por testes unitários.
RNF08	Licenciamento	Código e ativos próprios; dependências declaradas.

7. Arquitetura do projeto

7.1 Organização em módulos



Regra estrutural: `core` mantém somente o loop e seleciona cenas. A cena jogável coordena entidades, mundo e sistemas. Os dados da fase não conhecem renderização; o componente de vida não conhece jogador ou inimigo. Essa separação reduz acoplamento e facilita novas fases.

7.2 Responsabilidades

Módulo	Entrada	Saída
Game	Eventos, teclado, tempo	Atualização e quadro renderizado
Scenes	Eventos e contexto do jogo	Menu, partida ou resultado
Player	Movimento, comandos, plataformas	Estado físico e novos projéteis
Enemy	Tempo e posição da personagem	Patrulha e projétil hostil
Level	Câmera	Plataformas e cenário desenhado
StageDefinition	Configuração declarativa	Plataformas, spawns, início e saída
Health	Dano ou cura	Vida limitada entre zero e máximo
Physics	Escalares e Δt	Valores integrados/limitados
HUD	Jogador e estado da partida	Interface desenhada

7.3 Estrutura das entidades



7.4 Relações e multiplicidade

Origem	Relação	Destino	Motivo
Player, Enemy, Item, Projectile	generalização	Entity	Compartilham posição, velocidade, imagem e retângulo.
Game	composição 1:1	Scene e Audio	Gerencia o ciclo da aplicação.
GameplayScene	composição 1:1	Player, Level e Hud	Existem durante uma partida.
GameplayScene	agregação 1:N	Enemy, Item, Projectile	Coleções mudam durante a partida.
Player e Enemy	composição 1:1	Health	Compartilham as regras de vida sem duplicação.
Player e Enemy	criação	Projectile	Ambos disparam com regras distintas.
Player	dependência	Physics e Dynamics	Usa fórmulas, movimento e colisão modularizados.
Item	associação	Player	A coleta altera vida ou pontuação.